Budowa serwera EKG w Go

https://gemini.google.com/app/579a276af869a8af

User prompt: EKG_Serwer w GoLang Jesteś moim asystentem - nie zastępujesz mnie w pracy Naszym zadaniem jest stworzyć serwer odbierający paczki z pomiarami EKG przy pomocy Goroutines oraz websocket. Na początek pomóż mi skonfigurować środowisko i napisać coś podstawowego co będzie można rozbudowywać o kolejne funkcjonalnści Programuję w vscodium oraz nie będę korzystał z żadnego podjeścia agentowego. Każda linijka kodu powinna być przez mnie w pełni zrozumiana i jak najlepiej napisana

Response: Cześć! Bardzo mi odpowiada takie podejście. Będę Twoim wsparciem – wytłumaczę Ci mechanizmy, podrzucę fragmenty kodu, ale to Ty będziesz mieć pełną kontrolę i zrozumienie nad projektem.

Na początek przygotujemy środowisko, a następnie napiszemy absolutne minimum: serwer HTTP, który potrafi "podnieść" połączenie (upgrade) do protokołu WebSocket i w nieskończonej pętli nasłuchiwać przychodzących danych.

Krok 1: Konfiguracja środowiska w VSCodium

1. **Zainstaluj Go**: Upewnij się, że masz zainstalowane Go z golang.org.
2. **Rozszerzenie VSCodium**: W VSCodium zainstaluj oficjalne rozszerzenie **Go** (często widnieje jako "Go Team at Google"). Po instalacji może poprosić o doinstalowanie narzędzi (np. `gopls` do analizy kodu) – warto się na to zgodzić.
3. **Inicjalizacja projektu**: Otwórz terminal w pustym folderze, w którym chcesz trzymać projekt, i wykonaj polecenie:

   ```
   go mod init ekg_server
   ```

   To utworzy plik `go.mod`, który będzie zarządzał zależnościami.
4. **Pobranie biblioteki do WebSocketów**: Choć Go ma świetną bibliotekę standardową do sieci, najpopularniejszym i najlepiej przetestowanym pakietem do WebSocketów jest `gorilla/websocket`. Pobierz go:

   ```
   go get github.com/gorilla/websocket
   ```

Krok 2: Kod podstawowego serwera (plik `main.go`)

Utwórz plik `main.go`. Poniżej znajduje się najprostszy, w pełni zakomentowany kod. Celowo użyłem polskojęzycznych komentarzy, żeby wszystko było w 100% jasne.

```
package main

import (
	"fmt"
	"log"
	"net/http"

	"github.com/gorilla/websocket"
)

// Krok A: Konfiguracja "Upgradera"
// Upgrader służy do zmiany zwykłego żądania HTTP w stałe połączenie WebSocket.
var upgrader = websocket.Upgrader{
	ReadBufferSize:  1024,
	WriteBufferSize: 1024,
	// Na początek pozwalamy na połączenia z dowolnego źródła.
	// Kiedy będziesz pisać klienta (np. w przeglądarce), unikniesz problemów z CORS.
	CheckOrigin: func(r *http.Request) bool {
		return true
	},
}

// Krok B: Funkcja obsługująca pojedyncze połączenie
func handleECGConnection(w http.ResponseWriter, r *http.Request) {
	// 1. Upgrade połączenia do WebSocket
	ws, err := upgrader.Upgrade(w, r, nil)
	if err != nil {
		log.Println("Błąd podczas upgrade'u połączenia:", err)
```

```
			return // Jeśli się nie udało, kończymy obsługę tego klienta
		}

		// defer upewnia się, że połączenie zostanie zamknięte,
		// gdy funkcja handleECGConnection zakończy swoje działanie.
		defer ws.Close()

		log.Printf("Nowy klient podłączony z adresu: %s", r.RemoteAddr)

		// 2. Nieskończona pętla - serce naszej Goroutine
		for {
			// ReadMessage blokuje wykonanie pętli, dopóki nie nadejdzie nowa paczka danych.
			messageType, payload, err := ws.ReadMessage()
			if err != nil {
				log.Println("Klient rozłączył się lub wystąpił błąd odczytu:", err)
				break // Wychodzimy z pętli, co aktywuje defer ws.Close()
			}

			// Na ten moment po prostu wypisujemy zawartość paczki w konsoli.
			// %s zamienia tablicę bajtów (payload) na string.
			fmt.Printf("Otrzymano paczkę EKG (%d bajtów): %s\n", len(payload), string(payload))

			// Opcjonalnie: możemy odesłać klientowi potwierdzenie, że serwer przetrawił dane.
			ackMessage := []byte("Serwer: Paczka EKG odebrana.")
			err = ws.WriteMessage(messageType, ackMessage)
			if err != nil {
				log.Println("Błąd wysyłania potwierdzenia:", err)
				break
			}
		}
}

// Krok C: Główna funkcja uruchamiająca serwer
func main() {
		// Przypisujemy nasz endpoint "/ws" do funkcji obsługującej
		http.HandleFunc("/ws", handleECGConnection)

		fmt.Println("Serwer EKG nasłuchuje na porcie :8080...")

		// Uruchamiamy serwer HTTP.
		// WAŻNE: Funkcja ListenAndServe z biblioteki 'net/http' działa tak, że
		// DLA KAŻDEGO nowego zapytania automatycznie tworzy nową "Goroutine".
		// Dzięki temu tysiące klientów może podłączyć się pod "/ws" równocześnie.
		err := http.ListenAndServe(":8080", nil)
		if err != nil {
			log.Fatal("Krytyczny błąd serwera: ", err)
		}
}
```

Dlaczego to działa współbieżnie (Goroutines)?

Zauważ, że w kodzie nie napisaliśmy słowa kluczowego `go`, które zazwyczaj uruchamia *Goroutine* w GoLang. Dzieje się tak dlatego, że funkcja `http.ListenAndServe` pod spodem robi to za nas. Kiedy nowy pacjent/urządzenie uderza pod adres ws://localhost:8080/ws, serwer standardowy wywołuje `handleECGConnection` jako osobną gorutynę. Dzięki temu jedna długa pętla `for` w funkcji nie blokuje obsługi innych pacjentów.

Jak przetestować?

1. Zapisz plik `main.go`.
2. W terminalu uruchom serwer:

   ```
   go run main.go
   ```

3. Ponieważ nie mamy jeszcze klienta, możesz go zastąpić dodatkiem do przeglądarki (np. **Pie Socket**, **Postman** z funkcją WebSocket lub stroną typu wscat).

4. Połącz się pod adres: `ws://localhost:8080/ws`
5. Wyślij dowolną wiadomość tekstową (np. `{"id": 1, "lead_v1": 1.25, "lead_v2": 0.85}`) - powinieneś zobaczyć log w VSCodium i odpowiedź w kliencie.

Co robimy dalej?

Gdy upewnisz się, że rozumiesz ten kod i potrafisz zestawić połączenie, musimy zastanowić się nad budową paczki EKG.

- Jakie dane urządzenia będą do nas wysyłać? (JSON? Binarne protobufy?)
- Co serwer ma z nimi zrobić po odebraniu (tylko zapisywać do logu, zapisywać do bazy danych, czy przesyłać na żywo na jakiś dashboard dla lekarza)?

Daj znać, jak poszło z uruchomieniem!

User prompt: jak będzie wyglądała struktura takiego repo, gdzie będzie plik main a gdzie póxnijesze serwisy?

Response: W Go powszechnie stosuje się podział katalogów ułatwiający skalowanie i oddzielający punkt startowy aplikacji od logiki biznesowej. Zamiast trzymać wszystko w głównym folderze, warto od razu przyjąć strukturę opartą na sprawdzonych konwencjach.

Drzewo projektu

```
ekg_server/
├── cmd/
│   └── server/
│       └── main.go        // Punkt wejścia do aplikacji
├── internal/
│   ├── websocket/
│   │   └── handler.go     // Logika połączeń i upgradera (zależności sieciowe)
│   ├── ecg/
│   │   ├── parser.go      // Parsowanie paczek (np. bajty na struktury)
│   │   └── models.go      // Definicje struktur danych pomiarów EKG
│   └── storage/
│       └── repository.go  // Przyszły zapis odczytów na dysk lub do bazy
├── go.mod
└── go.sum
```

Katalog cmd/ Tutaj znajduje się wyłącznie plik `main.go`. Jego zadaniem jest wczytanie środowiska (porty, konfiguracja), powiązanie ze sobą poszczególnych serwisów i uruchomienie serwera `http.ListenAndServe`. Nie umieszcza się w nim logiki biznesowej. Jeśli w przyszłości napiszesz dodatkowy program do generowania sztucznych danych testowych, wyląduje on np. w `cmd/simulator/main.go`.

Katalog internal/ To mechanizm wbudowany w kompilator Go. Kod znajdujący się wewnątrz folderu `internal/` nie może być importowany przez zewnętrzne projekty (działa jak hermetyzacja dla całego repozytorium). Tutaj umieścisz wszystkie przyszłe serwisy, dzieląc je na niezależne od siebie pakiety:

- **internal/websocket**: Izoluje infrastrukturę WebSocket. Pliki tutaj nie muszą wiedzieć, czym jest EKG – mają jedynie utrzymać połączenie, odebrać bajty i przekazać je dalej.
- **internal/ecg**: Reprezentuje logikę medyczną i biznesową. Pakiet ten odbierze surowe dane od websocketa, zwaliduje je pod kątem błędów sprzętowych i zmapuje na konkretne struktury w Go (np. częstotliwość próbkowania, wartości z elektrod).
- **internal/storage**: Izoluje warstwę zapisu. Dzięki temu logika `ecg` nie musi wiedzieć, czy zapisujesz dane do bazy PostgreSQL, bazy szeregów czasowych jak InfluxDB, czy po prostu do plików tekstowych.

Rozbicie dotychczasowego kodu polegałoby na utworzeniu tej struktury, przeniesieniu funkcji `handleECGConnection` do pliku w `internal/websocket/` (ze zmienioną nazwą pakietu na `package websocket`) i wywołaniu jej w `main.go`. W jakim formacie (JSON, czysty tekst, dane binarne) planujesz początkowo przesyłać odczyty do serwera?

User prompt: Jakie zmiany będą w pliku man=in.go kiedy zmienia się drzewo katalogów?

Response: Po rozbiciu projektu na pakiety w folderze `internal/`, plik `main.go` ulegnie znacznemu odchudzeniu. Jego rola sprowadzi się wyłącznie do **importowania własnych modułów**, **inicjalizacji konfiguracji** oraz **uruchomienia serwera**.

Oto jak będzie wyglądał plik `cmd/server/main.go` po przeprowadzeniu tej refaktoryzacji:

```
package main

import (
	"fmt"
	"log"
	"net/http"

	// Importujemy nasz własny pakiet z katalogu internal.
	// Ścieżka importu to: <nazwa_modułu_z_go.mod>/internal/websocket
	"ekg_server/internal/websocket"
)

func main() {
	// 1. Tworzymy instancję naszego handlera WebSocket z pakietu internal/websocket
	wsHandler := websocket.NewHandler()

	// 2. Rejestrujemy metodę handlera dla endpointu /ws
	http.HandleFunc("/ws", wsHandler.HandleConnection)

	fmt.Println("Serwer EKG nasłuchuje na porcie :8080...")

	// 3. Uruchamiamy serwer HTTP
	err := http.ListenAndServe(":8080", nil)
	if err != nil {
		log.Fatal("Krytyczny błąd serwera: ", err)
	}
}
```

Kluczowe zmiany w `main.go` w stosunku do pierwszej wersji:

1. **Brak definicji `upgrader` oraz logiki połączenia:** Zniknęły zmienne i funkcje odpowiedzialne za proces podnoszenia połączenia (upgrade HTTP -> WebSocket) oraz pętla odczytu. Cała ta logika trafia do pakietu `internal/websocket`.
2. **Własny import (`ekg_server/internal/websocket`):** Do sekcji `import` dochodzi ścieżka do Twojego lokalnego pakietu. Przed `internal/...` podajesz nazwę modułu zdefiniowaną w pliku `go.mod` (w naszym przykładzie to `ekg_server`).
3. **Użycie eksportowanych funkcji/konstruktorów:** Zamiast przekazywać lokalną funkcję `handleECGConnection`, przekazujemy metodę `wsHandler.HandleConnection`.

Jak wygląda odpowiadający temu plik `internal/websocket/handler.go`?

Aby powyższy `main.go` zadziałał, w pliku `internal/websocket/handler.go` musisz zdefiniować strukturę i metody rozpoczynające się od **wielkiej litery** (w Go wielka litera oznacza, że element jest publiczny/eksportowany poza pakiet):

```
package websocket

import (
	"fmt"
	"log"
	"net/http"

	"github.com/gorilla/websocket"
)

// Handler strukturyzuje logikę WebSocketów i pozwala w przyszłości
// wstrzykiwać tu zależności (np. serwisy bazy danych, kanały Go).
type Handler struct {
	upgrader websocket.Upgrader
}
```

```
// NewHandler to tzw. konstruktor tworzący gotową instancję Handlera.
func NewHandler() *Handler {
	return &Handler{
		upgrader: websocket.Upgrader{
			ReadBufferSize:  1024,
			WriteBufferSize: 1024,
			CheckOrigin: func(r *http.Request) bool {
				return true
			},
		},
	}
}

// HandleConnection – wielka litera 'H' sprawia, że funkcja jest publiczna
// i widoczna dla pliku main.go.
func (h *Handler) HandleConnection(w http.ResponseWriter, r *http.Request) {
	ws, err := h.upgrader.Upgrade(w, r, nil)
	if err != nil {
		log.Println("Błąd podczas upgrade'u połączenia:", err)
		return
	}
	defer ws.Close()

	log.Printf("Nowy klient podłączony z adresu: %s", r.RemoteAddr)

	for {
		messageType, payload, err := ws.ReadMessage()
		if err != nil {
			log.Println("Klient rozłączył się lub wystąpił błąd odczytu:", err)
			break
		}

		fmt.Printf("Otrzymano paczkę EKG (%d bajtów): %s\n", len(payload), string(payload))

		ackMessage := []byte("Serwer: Paczka EKG odebrana.")
		err = ws.WriteMessage(messageType, ackMessage)
		if err != nil {
			log.Println("Błąd wysyłania potwierdzenia:", err)
			break
		}
	}
}
```

Jak teraz uruchomić projekt?

Gdy plik `main.go` znajduje się w katalogu `cmd/server/main.go`, uruchamiasz go z głównego folderu projektu poleceniem:

```
go run ./cmd/server
```

lub wskazując bezpośrednio plik:

```
go run cmd/server/main.go
```

User prompt: okej, na razie nie ma internal, w ogóle, ufamy w działanie githubowego websocket, jak mam zmodyfikować main.go które jest w cmd/server aby uruchomić przykładowy kod?

Response: Rozumiem! Czasami warto zrobić krok w tył i zacząć od jednego działającego pliku, żeby dobrze poczuć, jak to wszystko działa, zanim zaczniemy budować rozbudowaną architekturę.

Jeśli nie używamy jeszcze folderu `internal`, a cały nasz kod ma znajdować się w pliku `main.go` zlokalizowanym w `cmd/server/`, struktura Twojego projektu wygląda teraz tak:

```
ekg_server/
├── cmd/
```

```
│   └── server/
│       └── main.go
├── go.mod
└── go.sum
```

Zawartość pliku `cmd/server/main.go`

W tym pliku umieszczamy dokładnie to, co napisaliśmy za pierwszym razem. Kod ma wszystko w sobie: inicjalizację HTTP, konfigurację paczki `gorilla/websocket` oraz obsługę pętli.

Wklej poniższy kod do pliku `cmd/server/main.go`:

```
package main

import (
	"fmt"
	"log"
	"net/http"

	// Ufamy i korzystamy bezpośrednio z zewnętrznej biblioteki
	"github.com/gorilla/websocket"
)

// Konfiguracja "Upgradera" (zmiana HTTP na WebSocket)
var upgrader = websocket.Upgrader{
	ReadBufferSize:  1024,
	WriteBufferSize: 1024,
	CheckOrigin: func(r *http.Request) bool {
		return true // Zezwalamy na połączenia z dowolnego źródła
	},
}

// Funkcja obsługująca pojedyncze połączenie WebSocket
func handleECGConnection(w http.ResponseWriter, r *http.Request) {
	// Upgrade połączenia
	ws, err := upgrader.Upgrade(w, r, nil)
	if err != nil {
		log.Println("Błąd podczas upgrade'u połączenia:", err)
		return
	}
	defer ws.Close()

	log.Printf("Nowy klient podłączony z adresu: %s", r.RemoteAddr)

	// Główna pętla odbierająca paczki
	for {
		messageType, payload, err := ws.ReadMessage()
		if err != nil {
			log.Println("Klient rozłączył się lub wystąpił błąd odczytu:", err)
			break
		}

		fmt.Printf("Otrzymano paczkę EKG (%d bajtów): %s\n", len(payload), string(payload))

		// Odsyłamy potwierdzenie do klienta
		ackMessage := []byte("Serwer: Paczka EKG odebrana.")
		err = ws.WriteMessage(messageType, ackMessage)
		if err != nil {
			log.Println("Błąd wysyłania potwierdzenia:", err)
			break
		}
	}
}

func main() {
	// Przypinamy endpoint "/ws" do naszej funkcji
	http.HandleFunc("/ws", handleECGConnection)
```

```
	fmt.Println("Serwer EKG nasłuchuje na porcie :8080...")

	// Start serwera HTTP (pod spodem automatycznie używa Goroutines dla nowych żądań)
	err := http.ListenAndServe(":8080", nil)
	if err != nil {
		log.Fatal("Krytyczny błąd serwera: ", err)
	}
}
```

Jak to teraz uruchomić?

Ponieważ plik `main.go` nie leży już w głównym katalogu, musisz powiedzieć kompilatorowi Go, gdzie ma go szukać.

Otwórz terminal w głównym folderze Twojego projektu (czyli w folderze `ekg_server`, tam gdzie leży plik `go.mod`) i wpisz jedną z poniższych komend:

Opcja 1 (wskazanie dokładnej ścieżki do pliku):

```
go run cmd/server/main.go
```

Opcja 2 (wskazanie całego katalogu zawierającego pakiet main):

```
go run ./cmd/server
```

Obie komendy zrobią dokładnie to samo – skompilują kod i uruchomią Twój serwer. Od tego momentu możesz znów testować połączenie za pomocą dowolnego klienta WebSocket pod adresem `ws://localhost:8080/ws`.

Daj znać, jak tylko uda Ci się odebrać pierwszą testową paczkę! Wtedy zastanowimy się nad tym, jak chcemy dekodować informacje o pracy serca (np. ze stringa na strukturę w Go).
